

# docker文件

| 文件                        | 作用                                                              |
|---------------------------|-----------------------------------------------------------------|
| backend/Dockerfile        | 后端多阶段构建：Maven 编译 → JRE 运行，一份镜像含 platform-api + admin-api 两个 jar |
| admin-lite/Dockerfile     | 管理后台：Node 构建 → Nginx 静态服务                                       |
| uni-app/Dockerfile        | H5 用户端：Node 构建（含内存优化）→ Nginx 静态服务                               |
| admin-lite/nginx.conf     | SPA 路由 fallback + /api 反代到 admin-api:8081                       |
| uni-app/nginx.conf        | SPA fallback + /api + /ws WebSocket 反代到 platform-api:8080       |
| backend/.dockerignore     | 排除 target/、.mvn/ 等                                              |
| admin-lite/.dockerignore  | 排除 node_modules/、dist/ 等                                        |
| uni-app/.dockerignore     | 排除 node_modules/、dist/、日志等                                      |
| deploy/docker-compose.yml | 完整编排：7 个服务一条命令启动                                                |

## 架构说明

- **后端**：两个 Spring Boot 应用（platform-api:8080、admin-api:8081）共用一个镜像，通过 `command` 字段切换启动哪个 jar
- **前端**：Nginx 反代 /api 到后端容器名，容器间走 Docker 内部网络
- **基础设施**：PostgreSQL (pgvector) + Redis + MinIO，带健康检查，后端等基础设施就绪后启动
- **阿里云 Maven 镜像**：直接写入后端 Dockerfile，国内构建速度快

## 使用方法

```
# 进入 deploy 目录
cd D:\1\ai-app\app1\deploy

# 一键构建并启动全部服务
docker compose up -d --build

# 查看日志
docker compose logs -f platform-api

# 停止
docker compose down
```

启动后访问地址：

- 用户端 H5: `http://localhost:5180`

- 管理后台: `http://localhost:5175`
- platform-api: `http://localhost:8080`
- admin-api: `http://localhost:8081`
- MinIO 控制台: `http://localhost:9001`

⚠️ `docker-compose.yml` 中的 `JWT_SECRET` 和 `AI_MASTER_KEY` 是占位值, 生产环境请替换为强随机密钥。